A Simple Batched Threshold Encryption Scheme

Guru-Vamsi Policharla
Commonware, Inc.

1 Introduction

In this note, we construct a simple batched threshold encryption scheme that satisfies censorship resistance, does not suffer from epoch restrictions, and has quasi-linear decryption complexity in the batch size with a CPA ciphertext size of $|\mathbb{G}_1| + |\mathbb{G}_T|$. Using standard techniques in the random oracle model, one can handle arbitrary length messages with a ciphertext size of $|\mathbb{G}_1| + |\mathsf{msg}|$. CCA security can be obtained by appending a simulation-extractable NIZK proof of well-formedness of the ciphertext, which increases the ciphertext size by $2|\mathbb{F}|$. We prove security under the Decisional Bilinear q-Power Diffie-Hellman Assumption [BGW05] in asymmetric pairing groups, assuming the NIZK is zero-knowledge and straight-line simulation-extractable. These improvements come at the cost of an interactive setup phase (involving secure multiplications) and secret keys that grow linearly with the batch size. See Table 1 for a detailed comparison. For a discussion on the relevance of this primitive to encrypted mempools and MEV, we refer the reader to [CGPP24] and follow-up works (Section 4). We direct the reader to Section 5 for an overview of the construction.

2 Preliminaries

We use $[n]$ to denote the set $\{1, 2, \ldots, n\}$. The security parameter is denoted by $\lambda \in \mathbb{N}$. A function $f : \mathbb{N} \to \mathbb{N}$ is said to be polynomial if there exists a constant c such that $f(n) \leq n^c$ for all $n \in \mathbb{N}$, and we write $\mathsf{poly}(\cdot)$ to denote such a function. A function $f : \mathbb{N} \to [0, 1]$ is said to be negligible if for every $c \in \mathbb{N}$, there exists $N \in \mathbb{N}$ such that for all $n > N$, $f(n) < n^{-c}$, and we write $\mathsf{negl}(\cdot)$ to denote such a function. A probability is *noticeable* if it is not negligible, and *overwhelming* if it is equal to $1 - \mathsf{negl}(\lambda)$ for some negligible function $\mathsf{negl}(\lambda)$. For a set $\mathcal{S}$, we write $s \leftarrow \mathcal{S}$ to indicate that s is sampled uniformly at random from $\mathcal{S}$. An algorithm $\mathcal{A}$ is PPT (probabilistic polynomial-time) if its running time is bounded by some polynomial in the size of its input.

Bilinear groups. Let $\mathsf{GrpGen}(1^\lambda)$ output a description $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, \circ)$ where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are groups of prime order p, g_1, g_2 are generators of $\mathbb{G}_1, \mathbb{G}_2$ respectively, and $\circ : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is an efficiently computable bilinear map satisfying (i) *bilinearity*: $[x]_1 \circ [y]_2 = [xy]_T$ for all $x, y \in \mathbb{Z}_p$, and (ii) *non-degeneracy*: $g_1 \circ g_2 \neq 1_{\mathbb{G}_T}$. We write $g_T := g_1 \circ g_2$ and, for $b \in \{1, 2, T\}$, $[x]_b := x \cdot g_b$. We work in the Type-III (asymmetric) setting: no efficiently computable isomorphism between $\mathbb{G}_1$ and $\mathbb{G}_2$ is assumed.

Non-interactive Zero-Knowledge proofs. Let $\mathcal{L}$ be an NP-language and $\mathcal{R}$ the corresponding NP-relation. A Simulation Extractable-NIZK for $\mathcal{R}$ consists of the following algorithms:

- $\mathsf{Setup}(1^\lambda) \to (\mathsf{crs}, \mathsf{td})$: Takes as input a security parameter, and outputs a common reference string crs and trapdoor td.
- $\mathsf{Prove}(\mathsf{crs}, x, w) \to \pi$: Takes as input crs and any pair $(x, w) \in \mathcal{R}$ and outputs a proof π for the statement $x \in \mathcal{L}$.
- $\mathsf{Verify}(\mathsf{crs}, x, \pi) \to b$: Takes as input crs, statement x and proof π and outputs a bit b indicating whether verification has passed or failed.

- $\mathsf{SimProve}(\mathsf{crs}, \mathsf{td}, x) \to \pi$: Takes as input crs, trapdoor td and statement x and outputs a *simulated* proof π.

We will drop the crs term wherever implicit. A SE-NIZK satisfies the following properties:

- *Perfect Completeness.* An SE-NIZK satisfies perfect completeness if for any $(x, w) \in \mathcal{R}$, we have

$$\Pr\left[\begin{array}{c}(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda) \\ \pi \leftarrow \mathsf{Prove}(\mathsf{crs}, x, w)\end{array} : \mathsf{Verify}(x, \pi) = 1\right] = 1.$$

- *Proof of knowledge (and soundness).* For every PPT adversary $\mathcal{A}$, there is a PPT extractor Ext such that

$$\Pr\left[\begin{array}{c}(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}(\mathsf{crs}) \\ w \leftarrow \mathsf{Ext}(\mathsf{crs}, x, \pi)\end{array} : \begin{array}{c}\mathsf{Verify}(x, \pi) = 1 \\ (x, w) \notin \mathcal{R}\end{array}\right] \leq \mathsf{negl}(\lambda).$$

- *Computational Zero-knowledge.* There exists an algorithm SimProve such that for all PPT adversaries $\mathcal{A}$:

$$\left|\Pr\left[\begin{array}{c}(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda) \\ (x, w) \leftarrow \mathcal{A}(\mathsf{crs}) \\ \pi \leftarrow \mathsf{Prove}(\mathsf{crs}, x, w)\end{array} : \begin{array}{c}(x, w) \in \mathcal{R} \\ \mathcal{A}(\pi) = 1\end{array}\right] - \Pr\left[\begin{array}{c}(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda) \\ (x, w) \leftarrow \mathcal{A}(\mathsf{crs}) \\ \pi \leftarrow \mathsf{SimProve}(\mathsf{crs}, \mathsf{td}, x)\end{array} : \begin{array}{c}(x, w) \in \mathcal{R} \\ \mathcal{A}(\pi) = 1\end{array}\right]\right| \leq \mathsf{negl}(\lambda).$$

- *Simulation-Extractability.* For every PPT adversary $\mathcal{A}$, there exists a PPT extractor Ext such that

$$\Pr\left[\begin{array}{c}(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}^{\mathsf{SimProve}(\mathsf{crs}, \mathsf{td}, \cdot)}(\mathsf{crs}) \\ w \leftarrow \mathsf{Ext}(\mathsf{crs}, x, \pi)\end{array} : \begin{array}{c}\mathsf{Verify}(x, \pi) = 1 \\ \wedge (x, w) \notin \mathcal{R} \\ \wedge (x, \pi) \notin Q\end{array}\right] \leq \mathsf{negl}(\lambda).$$

  where $\mathcal{A}$ has oracle access to $\mathsf{SimProve}(\mathsf{crs}, \mathsf{td}, \cdot)$, and Q is the list of statement-proof pairs returned by that oracle.

We will also demand that the proof is straight-line extractable. This means that the extractor can, on input a valid proof and (potentially) the list of random-oracle queries made by the adversary (prover), extract a witness with overwhelming probability without rewinding the prover. When the SE-NIZK is instantiated in the random-oracle model, the crs may be taken as empty and extraction uses the adversary's random-oracle queries.

3 Definitions

We recall the definitions of batched threshold encryption.

Definition 1 (Batched Threshold Encryption)**.** *A* batched threshold encryption (BTE) *scheme instantiated with $N \in \mathbb{N}$ parties, a threshold of $t \in [N]$, maximum batch size of $B \in \mathbb{N}$, and message space of $\mathcal{M}$ consists of the following algorithms:*

- $\mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t) \to (\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk})$*: The setup algorithm takes the security parameter λ, the maximum batch size B, the number of parties N, and the threshold t, and outputs a public encryption key* ek*, a public decryption key* dk*, and N secret keys* $(\mathsf{sk}_1, \ldots, \mathsf{sk}_N)$.

- $\mathsf{Enc}(\mathsf{ek}, m) \to \mathsf{ct}$*: The encryption algorithm takes the encryption key* ek *and a message $m \in \mathcal{M}$, and outputs a ciphertext* ct.

- $\mathsf{PartialDec}(\mathsf{sk}_j, \{\mathsf{ct}_i\}_{i\in[B]}) \to \mathsf{pd}_j$*: The (deterministic) partial-decryption algorithm takes a secret key* sk_j *for a party* $j \in [N]$ *and* B *ciphertexts, and outputs a partial decryption share* pd_j *or* $\perp$*.*

- $\mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j, \{\mathsf{ct}_i\}_{i\in[B]}) \to \{0,1\}$*: The (deterministic) verification algorithm takes the decryption key* dk*, a party index* $j \in [N]$*, a claimed partial decryption share* pd_j*, and* B *ciphertexts, and outputs* 1 *iff* pd_j *is a valid share for party* j *on that batch.*

- $\mathsf{Combine}(\mathsf{dk}, T \subseteq [N], \{\mathsf{pd}_j\}_{j\in T}) \to \mathsf{pd}$*: The (deterministic) combiner algorithm takes the decryption key* dk*, a subset* $T \subseteq [N]$*, and the partial decryption shares of these* $|T|$ *parties, and outputs a partial decryption* pd *or* $\perp$*.*

- $\mathsf{Dec}(\mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]}) \to \{m_i\}_{i\in[B]}$*: The (deterministic) decryption algorithm takes the decryption key* dk*, a partial decryption* pd *and* B *ciphertexts, and outputs the corresponding messages, where each* $m_i \in \mathcal{M} \cup \{\perp\}$*.*

Let $\mathsf{BTE} = (\mathsf{Setup}, \mathsf{Enc}, \mathsf{PartialDec}, \mathsf{Verify}, \mathsf{Combine}, \mathsf{Dec})$ be a BTE scheme.

Definition 2 (Correctness of BTE)**.** *We say* BTE *is* correct *if for all* $\lambda, B, N, t \in \mathbb{N}$ *with* $t \le N$*, all* $T \subseteq [N]$ *such that* $|T| \ge t$*, and* $\{m_i\}_{i\in[B]} \in \mathcal{M}^B$*,*

$$\Pr\left[\begin{array}{c|l} \mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j, \{\mathsf{ct}_i\}_{i\in[B]}) = 1\ \forall j \in T & (\mathsf{ek}, \{\mathsf{sk}_j\}_{j\in[N]}, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t) \\ \quad \wedge\ \mathsf{Dec}(\mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]}) = \{m_i\}_{i\in[B]} & \mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{ek}, m_i)\ \forall i \in [B] \\ & \mathsf{pd}_j \leftarrow \mathsf{PartialDec}(\mathsf{sk}_j, \{\mathsf{ct}_i\}_{i\in[B]})\ \forall j \in T \\ & \mathsf{pd} \leftarrow \mathsf{Combine}(\mathsf{dk}, T, \{\mathsf{pd}_j\}_{j\in T}) \end{array}\right] = 1.$$

Definition 3 (B-IND-CCA Security of BTE)**.** BTE *is* B-IND-CCA-secure *if all non-uniform PPT adversaries* $\mathcal{A}$ *have negligible advantage in the following game:*

- *The challenger runs* $(\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t)$ *and sends* ek *and* dk *to* $\mathcal{A}$*. It then receives a subset* $S \subseteq [N]$ *of size* $t-1$ *from* $\mathcal{A}$ *and responds with* $\{\mathsf{sk}_j\}_{j\in S}$*.*

- ***Pre-challenge queries:*** *The adversary may issue an arbitrary number of queries for computing partial decryption shares:* $\mathcal{A}$ *sends a list* $\{\mathsf{ct}_i\}_{i\in[B]}$ *of* B *ciphertexts, and the challenger computes* $\mathsf{pd}_j := \mathsf{PartialDec}(\mathsf{sk}_j, \{\mathsf{ct}_i\}_{i\in[B]})$ *for* $j \in [N] \setminus S$*, and sends them to* $\mathcal{A}$*.*

- ***Challenge round:*** $\mathcal{A}$ *sends two messages* $m_0, m_1 \in \mathcal{M}$*. The challenger samples* $b \leftarrow\$ \{0,1\}$*, computes* $\mathsf{ct}^* \leftarrow \mathsf{Enc}(\mathsf{ek}, m_b)$*, and sends* ct^* *to* $\mathcal{A}$*.*

- ***Post-challenge queries:*** *The adversary can again issue an arbitrary number of queries for computing partial decryption shares. If* $\mathcal{A}$ *sends a list* $\{\mathsf{ct}_i\}_{i\in[B]}$ *such that* $\mathsf{ct}^* \in \{\mathsf{ct}_i\}_{i\in[B]}$*, then the challenger returns* $\perp$ *for that query.*

- ***Output:*** $\mathcal{A}$ *outputs a bit* $b' \in \{0,1\}$*.*

The adversary's advantage is

$$\mathsf{Adv}^{\mathsf{ind\text{-}cca}}_{\mathsf{BTE},\mathcal{A}}(\lambda, B, N, t) := \left|\Pr[b' = b] - \tfrac{1}{2}\right|.$$

Robustness. A BTE scheme must be robust against malicious decryption servers or clients that submit malformed ciphertexts or decryption queries. Informally, we require the following two properties:

- **Consistency:** It should be computationally infeasible for an adversary to produce two different sets of partial decryptions for the same batch of ciphertexts such that both sets verify yet Combine yields conflicting plaintexts.

- **Rogue Ciphertext Security:** It should be computationally infeasible for an adversary to craft a malformed ciphertext that causes decryption of an honest party's ciphertext in the same batch to fail.

Definition 4 (Rogue Ciphertext Security)**.** BTE *is* rogue ciphertext secure *if all non-uniform PPT adversaries $\mathcal{A}$ have a negligible probability of winning the following game:*

- *The challenger runs* $(\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t)$ *and sends everything to* $\mathcal{A}$.
- $\mathcal{A}$ *outputs a message* $m \in \mathcal{M}$.
- *The challenger computes* $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{ek}, m)$.
- $\mathcal{A}$ *outputs* B *ciphertexts* $\{\mathsf{ct}_i\}_{i\in[B]}$*, a subset* $S \subseteq [N]$ *with* $|S| = t$*, and partial decryption shares* $\{\mathsf{pd}_j\}_{j\in S}$.
- *The adversary wins the game if:*
  - *there exists* $i^* \in [B]$ *such that* $\mathsf{ct} = \mathsf{ct}_{i^*}$*,*
  - $\mathsf{Verify}(\mathsf{dk}, \ell, \mathsf{pd}_\ell, \{\mathsf{ct}_i\}_{i\in[B]}) = 1$ *for all* $\ell \in S$*,*
  - $\mathsf{pd} \leftarrow \mathsf{Combine}(\mathsf{dk}, S, \{\mathsf{pd}_j\}_{j\in S})$ *succeeds and* $(m'_1, \ldots, m'_B) \leftarrow \mathsf{Dec}(\mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]})$*,*
  - $m'_{i^*} = \perp$ *or* $m'_{i^*} \neq m$.

Definition 5 (Consistency of BTE)**.** BTE *is* consistent *if all non-uniform PPT adversaries $\mathcal{A}$ have a negligible probability of winning the following game:*

- *The challenger runs* $(\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t)$ *and sends everything to* $\mathcal{A}$.
- $\mathcal{A}$ *outputs* B *ciphertexts* $\{\mathsf{ct}_i\}_{i\in[B]}$*, two subsets* $S_1, S_2 \subseteq [N]$ *with* $|S_1| = |S_2| = t$*, and partial decryptions* $\{\mathsf{pd}^1_j\}_{j\in S_1}$ *and* $\{\mathsf{pd}^2_j\}_{j\in S_2}$.
- *For each* $b \in \{1, 2\}$*, define*

$$\mathsf{pd}^{(b)} \leftarrow \mathsf{Combine}(\mathsf{dk}, S_b, \{\mathsf{pd}^b_j\}_{j\in S_b}) \qquad \textit{and} \qquad \{m^{(b)}_i\}_{i\in[B]} \leftarrow \mathsf{Dec}(\mathsf{dk}, \mathsf{pd}^{(b)}, \{\mathsf{ct}_i\}_{i\in[B]}).$$

- *The adversary wins if:*
  - $\mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}^b_j, \{\mathsf{ct}_i\}_{i\in[B]}) = 1$ *for all* $b \in \{1, 2\}$ *and all* $j \in S_b$*,*
  - $\mathsf{pd}^{(1)} \neq \perp$ *and* $\mathsf{pd}^{(2)} \neq \perp$*,*
  - $\{m^{(1)}_i\}_{i\in[B]} \neq \{m^{(2)}_i\}_{i\in[B]}$.

4 Related Work

Batched Threshold Encryption (BTE) was introduced in [CGPP24]. While the initial work required an interactive setup protocol for each batch of ciphertexts decrypted, subsequent work [CGPW25, AFP25, SAS24, BFOQ25] removed this but came with additional caveats. [CGPW25, AFP25, SAS24] showed how to build a BTE scheme with a one-time setup but ciphertexts still had to be encrypted to a certain batch number (epoch). Failure to be included in the chosen epoch would result in the ciphertext never being decrypted. [FPTX25] partially addressed this by increasing the CRS size by a factor of k – which allows a user's ciphertext to be included in any of the k batches but at the end of k batches, the public key must be refreshed. Furthermore, censorship resistance comes at the cost of a more expensive $O(B \log^2 B)$ decryption complexity.

In a separate line of work, [BFOQ25] removes the epoch restriction but users must still choose a *position* in the batch. This leaves transactions vulnerable to censorship: an attacker can outbid a victim for a conflicting position and thereby block inclusion of the victim's transaction. Follow-up work [ABD+25] improved the decryption complexity to $O(B \log B)$ and also provided improvements over virtualization in the weighted setting but still suffered from the same censorship issues.

Finally, [BNRT26] provided a construction based on partial fraction techniques which avoids both the epoch restriction and censorship issues. However, this comes at the cost of an interactive setup (involving secure inversions) and the size of secret keys held by each party grows with the batch size.

Also worth noting are two works [BCF+25] and [GWWW25] that build batched threshold encryption and batched identity based encryption respectively, with silent setup.

Our work also requires an interactive setup phase (involving secure multiplications) and secret keys held by each party grow with the batch size. It has the shortest ciphertext amongst all known schemes $|\mathbb{G}_1| + |\mathbb{G}_T|$, supports quasi-linear decryption complexity $O(B \log B)$, and we believe it's a conceptually simpler construction.

Construction	$\lvert\mathsf{dk}\rvert$	$\lvert\mathsf{sk}\rvert$	Setup	$\lvert\mathsf{ct}\rvert$	Dec.	Epochless	CR	Model
Batch IBE [CGPW25], [SAS24], [AFP25]	$O(B)$	$O(1)$	DKG	$3[\mathbb{G}_1] + [\mathbb{G}_T]$	$O(B \log^2 B)$	✗	✓	GGM
BEAT-MEV [BFOQ25]	$O(B)$	$O(1)$	DKG	$3[\mathbb{G}_1] + [\mathbb{G}_T]$	$O(B^2)$	✓	✗	DBPDH
TrX [FPTX25]	$O(kB)$	$O(1)$	DKG	$2[\mathbb{G}_1] + [\mathbb{G}_T]$	$O(B \log^2 B)$	✓	✓	GGM
[GWWW25]	$O(B)$	$O(1)$	DKG	$2[\mathbb{G}_1] + [\mathbb{G}_T]$	$O(B^2)$	✗	✓	GGM
[ABD+25]	$O(B)$	$O(1)$	DKG	$2[\mathbb{G}_1] + [\mathbb{G}_T]$	$O(B \log B)$	✓	✗	q-DBPDH + AGM
[BNRT26][1]	$O(BN)$	$O(B)$	MPC	$2[\mathbb{G}_1] + [\mathbb{G}_T]$	$O(B \log B)$	✓	✓	q-StatRankDef
This work	$O(BN)$	$O(B)$	MPC	$[\mathbb{G}_1] + [\mathbb{G}_T]$	$O(B \log B)$	✓	✓	q-DBPDH

Table 1: Comparison of batched encryption systems based on elliptic curves. Here $|\mathsf{dk}|$ denotes the total size of decryption key (including the CRS), $|\mathsf{sk}|$ denotes the secret-key size (per party), and $|\mathsf{ct}|$ denotes the ciphertext size with the NIZK proof omitted. Setup indicates whether the protocol relies on distributed key generation (DKG) or multi-party computation (MPC). Dec indicates the decryption complexity. CR indicates censorship resistance, Epochless indicates that the scheme does not suffer from epoch restrictions. Model indicates the underlying hardness assumption or idealized model. The parameter B denotes the maximum batch size, N denotes the committee size, and in TrX k denotes the number of batches which can be decrypted before the public key has to be freshly sampled again.

5 Construction

For simplicity, we first describe the CPA-secure core of the construction for a single server. We then explain how to add CCA security via a simulation-extractable NIZK and how to thresholdize the scheme. The full construction is described formally in Fig. 1.

- A trusted party runs the setup protocol and publishes the encryption key together with the punctured powers-of-τ values needed for decryption:

$$\mathsf{ek} := \left[\tau^{B+1}\right]_T \qquad \text{and} \qquad \mathsf{dk} := \left(\{h_j := \left[\tau^j\right]_2\}_{j \in [2B] \setminus \{B+1\}}\right).$$

- Encryption is then just ElGamal-style ciphertext in $\mathbb{G}_T$:

$$\mathsf{ct} := \left([k]_1\,,\ m + \left[k\tau^{B+1}\right]_T\right).$$

- To decrypt a batch $\{\mathsf{ct}_i\}_{i \in [B]}$, let $\mathsf{ct}_i = (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2})$ with $\mathsf{ct}_{i,1} = [k_i]_1$. It is sufficient to publish:

$$\mathsf{pd} := \sum_{i \in [B]} \tau^i \cdot \mathsf{ct}_{i,1} = \left[\sum_{i \in [B]} k_i \tau^i\right]_1.$$

[1] Although the current version of the paper describes a construction with $O(B^2)$ decryption complexity, the authors have privately communicated to us that they have a more efficient decryption algorithm with $O(B \log B)$ complexity.

- To decrypt the i-th ciphertext, one can compute:

$$\left[k_i \tau^{B+1}\right]_T = \mathsf{pd} \circ h_{B+1-i} - \sum_{\substack{\ell \in [B] \\ \ell \neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i}.$$

  Subtracting this mask from $\mathsf{ct}_{i,2}$ recovers m_i.

- To make the scheme CCA-secure, we augment each ciphertext with a simulation-extractable NIZK proof for the relation $\{k \mid \mathsf{ct}_1 = [k]_1 \wedge \mathsf{ct}_2\}$.

- To thresholdize this scheme, we secret share $\{\tau^i\}_{i \in [B]}$ among the N servers. Concretely, let server j hold shares $\{\sigma_j^i\}_{i \in [B]}$ of $\{\tau^i\}_{i \in [B]}$ and return the partial decryption

$$\mathsf{pd}_j := \sum_{i \in [B]} \sigma_j^i \cdot \mathsf{ct}_{i,1}.$$

  To verify these shares publicly, we additionally publish $\{v_j^i := \left[\sigma_j^i\right]_2\}_{(i,j) \in [B] \times [N]}$ as part of the trusted setup. Then anyone can check a claimed partial decryption by checking whether

$$\mathsf{pd}_j \circ g_2 = \sum_{i \in [B]} \mathsf{ct}_{i,1} \circ v_j^i.$$

Correctness. For Verify: an honest partial decryption satisfies $\mathsf{pd}_j = \sum_{i \in [B]} \sigma_j^i \cdot \mathsf{ct}_{i,1}$, so $\mathsf{pd}_j \circ g_2 = \sum_{i \in [B]} \mathsf{ct}_{i,1} \circ \left[\sigma_j^i\right]_2 = \sum_{i \in [B]} \mathsf{ct}_{i,1} \circ v_j^i$, which is exactly the check performed by Verify.

For Combine: since each $\sigma_j^i = f_i(j)$ for a degree-$(t-1)$ polynomial f_i with $f_i(0) = \tau^i$, Lagrange interpolation gives $\mathsf{pd} = \sum_{j \in T} \lambda_{T,j} \cdot \mathsf{pd}_j = \sum_{j \in T} \lambda_{T,j} \sum_{i \in [B]} f_i(j) \cdot \mathsf{ct}_{i,1} = \sum_{i \in [B]} \tau^i \cdot \mathsf{ct}_{i,1}$, which is the value used by Dec.

Working out the algebra, we see that the decryption algorithm computes:

$$\begin{aligned}
\mathsf{pd} \circ h_{B+1-i} - \sum_{\substack{\ell \in [B] \\ \ell \neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i} &= \left[\sum_{\ell \in [B]} k_\ell \tau^\ell\right]_1 \circ \left[\tau^{B+1-i}\right]_2 - \sum_{\substack{\ell \in [B] \\ \ell \neq i}} [k_\ell]_1 \circ \left[\tau^{\ell+B+1-i}\right]_2 \\
&= \sum_{\ell \in [B]} \left[k_\ell \tau^{\ell+B+1-i}\right]_T - \sum_{\substack{\ell \in [B] \\ \ell \neq i}} \left[k_\ell \tau^{\ell+B+1-i}\right]_T \\
&= \left[k_i \tau^{i+B+1-i}\right]_T = \left[k_i \tau^{B+1}\right]_T,
\end{aligned}$$

which is exactly the mask used when encrypting the ith plaintext, since

$$\mathsf{ct}_{i,2} = \left[k_i \tau^{B+1}\right]_T + m_i.$$

Therefore

$$\mathsf{ct}_{i,2} - z_i = m_i + \left[k_i \tau^{B+1}\right]_T - \left[k_i \tau^{B+1}\right]_T = m_i,$$

which proves correctness.

Theorem 1 (Rogue ciphertext security)**.** *The construction in Fig. 1 satisfies rogue ciphertext security.*

Proof. Let $\mathcal{A}$ be an adversary in the game of Definition 4. Suppose it outputs a batch $\{\mathsf{ct}_i\}_{i \in [B]}$, a threshold set $S \subseteq [N]$, and shares $\{\mathsf{pd}_j\}_{j \in S}$ such that $\mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j, \{\mathsf{ct}_i\}_{i \in [B]}) = 1$ for every $j \in S$. Fix some $j \in S$. Since Verify accepts, every ciphertext proof is valid and

$$\mathsf{pd}_j \circ g_2 = \sum_{i \in [B]} \mathsf{ct}_{i,1} \circ v_j^i.$$

Batched Threshold Encryption

$\mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t)$:

- $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, \circ) \leftarrow \mathsf{GrpGen}(1^\lambda)$
- Sample $\tau \leftarrow \mathbb{Z}_p$
- Run the setup for the chosen NIZK and let crs denote the resulting public parameters, if any.
- For each $i \in [B]$, secret-share τ^i among N servers with threshold t: server j receives share σ_j^i
- Output:

$$\mathsf{ek} := \left(\left[\tau^{B+1}\right]_T, \mathsf{crs}\right)$$
$$\mathsf{dk} := \left(\{h_j := \left[\tau^j\right]_2\}_{j \in [2B] \setminus \{B+1\}},\ \{v_j^i := \left[\sigma_j^i\right]_2\}_{(i,j) \in [B] \times [N]},\ \mathsf{crs}\right)$$
$$\mathsf{sk}_j := \left(\{\sigma_j^i\}_{i \in [B]}, \mathsf{crs}\right) \quad \text{for each server } j \in [N]$$

$\mathsf{Enc}(\mathsf{ek}, m)$: where $m \in \mathbb{G}_T$

- Parse $\mathsf{ek} = (E, \mathsf{crs})$
- Sample $k \leftarrow \mathbb{Z}_p$ and set $\mathsf{ct}_1 := [k]_1$
- Set $\mathsf{ct}_2 := m + k \cdot E$
- Prove $\pi \leftarrow \left\{k \mid \mathsf{ct}_1 = [k]_1 \wedge \mathsf{ct}_2\right\}$
- Output $\mathsf{ct} := (\mathsf{ct}_1, \mathsf{ct}_2, \pi)$

$\mathsf{PartialDec}(\mathsf{sk}_j, \{\mathsf{ct}_i\}_{i \in [B]})$:

- Parse $\mathsf{sk}_j = (\{\sigma_j^i\}_{i \in [B]}, \mathsf{crs})$ and $\{\mathsf{ct}_i = (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}, \pi_i)\}_{i \in [B]}$
- If $\mathsf{NIZK.Verify}(\mathsf{crs}, (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}), \pi_i) = 0$ for some $i \in [B]$, output $\perp$
- Output $\mathsf{pd}_j := \sum_{i \in [B]} \sigma_j^i \cdot \mathsf{ct}_{i,1}$

$\mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j, \{\mathsf{ct}_i\}_{i \in [B]})$:

- Parse dk to recover $\{v_j^i\}_{(i,j) \in [B] \times [N]}$ and crs, and each $\mathsf{ct}_i = (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}, \pi_i)$.
- If $\mathsf{NIZK.Verify}(\mathsf{crs}, (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}), \pi_i) = 0$ for some $i \in [B]$, output 0. Otherwise, output 1 iff $\mathsf{pd}_j \circ g_2 = \sum_{i \in [B]} \mathsf{ct}_{i,1} \circ v_j^i$.

$\mathsf{Combine}(\mathsf{dk}, T, \{\mathsf{pd}_j\}_{j \in T})$: where $T \subseteq [N]$ and $|T| = t$

- Interpolate in the exponent to recover the pre decryption key: $\mathsf{pd} := \left[\sum_{i \in [B]} k_i \tau^i\right]_1$.

$\mathsf{Dec}(\mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i \in [B]})$:

- For each $i \in [B]$, compute the mask $z_i \in \mathbb{G}_T$:

$$z_i := \mathsf{pd} \circ h_{B+1-i} - \sum_{\substack{\ell \in [B] \\ \ell \neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i}$$

- For each $i \in [B]$: output $m_i := \mathsf{ct}_{i,2} - z_i \in \mathbb{G}_T$.

Figure 1: BTE scheme for N servers with threshold t, batch size B, and message space $\mathcal{M} = \mathbb{G}_T$.

Because $\mathbb{G}_1$ is cyclic, for each $i \in [B]$ there exists $k_i \in \mathbb{Z}_p$ such that $\mathsf{ct}_{i,1} = [k_i]_1$. Using $v_j^i = [\sigma_j^i]_2$, we get

$$\sum_{i\in[B]} \mathsf{ct}_{i,1} \circ v_j^i = \sum_{i\in[B]} [k_i]_1 \circ [\sigma_j^i]_2 = \sum_{i\in[B]} [k_i\sigma_j^i]_T = \Big(\sum_{i\in[B]} \sigma_j^i \cdot \mathsf{ct}_{i,1}\Big) \circ g_2.$$

By non-degeneracy of the pairing, this implies

$$\mathsf{pd}_j = \sum_{i\in[B]} \sigma_j^i \cdot \mathsf{ct}_{i,1},$$

so every accepted share is exactly the honest partial decryption share for server j on this batch.

Therefore $\mathsf{Combine}(\mathsf{dk}, S, \{\mathsf{pd}_j\}_{j\in S})$ reconstructs the same aggregate pd that would be obtained from honest partial decryption shares. Given that the Dec algorithm is deterministic, decrypting the same ciphertext batch with the same reconstructed partial decryption yields the same message vector. Moreover, Dec always outputs an element of $\mathbb{G}_T$ (never $\perp$), so the case $m'_{i^*} = \perp$ is also excluded. Therefore the adversary can never win the game. □

Theorem 2 (Consistency)**.** *The construction in Fig. 1 is consistent.*

Proof. Let $\mathcal{A}$ be an adversary in the game of Definition 5. Suppose it outputs a batch $\{\mathsf{ct}_i\}_{i\in[B]}$, two subsets $S_1, S_2 \subseteq [N]$ of size t, and shares $\{\mathsf{pd}_j^1\}_{j\in S_1}$ and $\{\mathsf{pd}_j^2\}_{j\in S_2}$ that satisfy the winning conditions. Since Verify accepts every submitted share, all ciphertext proofs are valid and, for each $b \in \{1, 2\}$ and each $j \in S_b$,

$$\mathsf{pd}_j^b \circ g_2 = \sum_{i\in[B]} \mathsf{ct}_{i,1} \circ v_j^i.$$

Because $\mathbb{G}_1$ is cyclic, for each $i \in [B]$ there exists a unique $k_i \in \mathbb{Z}_p$ such that $\mathsf{ct}_{i,1} = [k_i]_1$. Using $v_j^i = [\sigma_j^i]_2$, the same calculation as in the proof of Theorem 1 shows that every accepted share must equal the honest partial decryption share:

$$\mathsf{pd}_j^b = \sum_{i\in[B]} \sigma_j^i \cdot \mathsf{ct}_{i,1}.$$

By uniqueness of polynomial interpolation, combining either threshold set recovers the same value:

$$\mathsf{pd}^{(1)} = \mathsf{Combine}(\mathsf{dk}, S_1, \{\mathsf{pd}_j^1\}_{j\in S_1}) = \mathsf{pd} = \mathsf{Combine}(\mathsf{dk}, S_2, \{\mathsf{pd}_j^2\}_{j\in S_2}) = \mathsf{pd}^{(2)}.$$

Since Dec is deterministic, decrypting the same ciphertext batch with the same reconstructed partial decryption yields the same message vector. Hence

$$\{m_i^{(1)}\}_{i\in[B]} = \{m_i^{(2)}\}_{i\in[B]},$$

contradicting the adversary's winning condition. Therefore the adversary's winning probability is 0. □

To prove B-IND-CCA, we introduce the following hardness assumption.

Definition 6 (Decisional Bilinear q-Power Diffie-Hellman Assumption (q-DBPDH))**.** *Let* GrpGen *be an asymmetric bilinear group generator. The q-DBPDH problem is hard for* GrpGen *if the following two distributions are computationally indistinguishable:*

$$\mathcal{D}_0 = \Big(\{[\tau^i]_1\}_{i\in[q]},\ \{[\tau^i]_2\}_{i\in[2q]\setminus\{q+1\}},\ [k]_1,\ [k\tau^{q+1}]_T\Big),$$
$$\mathcal{D}_1 = \Big(\{[\tau^i]_1\}_{i\in[q]},\ \{[\tau^i]_2\}_{i\in[2q]\setminus\{q+1\}},\ [k]_1,\ [z]_T\Big),$$

where $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, \circ) \leftarrow \mathsf{GrpGen}(1^\lambda)$ *and* $\tau, k, z \leftarrow \mathbb{Z}_p$.

Remark 1. *Although we use an indistinguishability based assumption, one can use the weaker search variant by modifying the construction to use a hash based key derivation function to first derive a key from the* $\left[k\tau^{q+1}\right]_T$ *term via a random oracle and use that to mask the message.*

Theorem 3 (B-IND-CCA security)**.** *Assume the q-DBPDH assumption (Definition 6) and assume the NIZK used in the construction is zero-knowledge and straight-line simulation-extractable. Then the construction in Fig. 1 is B-IND-CCA-secure.*

Proof. Let $\mathcal{A}$ be an adversary that wins the B-IND-CCA game (Definition 3) with non-negligible probability. Consider the following hybrids:

$\mathbf{H}_1$: This is the real game with challenge bit $b = 0$, so the challenge ciphertext is

$$\mathsf{ct}^* = \left([k^*]_1\,,\ m_0 + \left[k^*\tau^{B+1}\right]_T\,,\ \pi^*\right),$$

where π^* is generated honestly.

$\mathbf{H}_2$: This is identical to $\mathbf{H}_1$, except that the proof on the challenge ciphertext is generated with the NIZK simulation algorithm. The hybrids $\mathbf{H}_1$ and $\mathbf{H}_2$ are computationally indistinguishable by the zero-knowledge property of the NIZK. At this point, the challenger no longer needs the scalar k^*; it only uses the group elements $[k^*]_1$ and $\left[k^*\tau^{B+1}\right]_T$.

$\mathbf{H}_3$: This is identical to $\mathbf{H}_2$, except that the second ciphertext component is replaced by $m_0 + U$, where $U \leftarrow \mathbb{G}_T$ is uniform:

$$\mathsf{ct}^* = \left([k^*]_1\,,\ m_0 + U,\ \pi^*\right).$$

We show that $\mathbf{H}_2$ and $\mathbf{H}_3$ are computationally indistinguishable provided the B-DBPDH Assumption (Definition 6) holds. Suppose there exists an adversary distinguishing this hybrid from the previous one. We build a reduction $\mathcal{B}$ that wins the indistinguishability game for the q-DBPDH Assumption.

The reduction is given:

$$\left(\{g_i\}_{i\in[B]},\ \{h_j\}_{j\in[2B]\setminus\{B+1\}},\ [k]_1\,,\ Z\right),$$

and must decide whether $Z = \left[k\tau^{B+1}\right]_T$ or uniformly random in $\mathbb{G}_T$. The reduction computes $\left[\tau^{B+1}\right]_T := g_B \circ h_1$, runs the NIZK setup and publishes the crs and retains the NIZK trapdoor td (if applicable). It also sets

$$\mathsf{ek} := \left(\left[\tau^{B+1}\right]_T, \mathsf{crs}\right).$$

The public verification keys in dk are defined below. The reduction receives a corruption set $S \subseteq [N]$ of size $t-1$ from $\mathcal{A}$.

For each slot $i \in [B]$ and each corrupted party $u \in S$, the reduction samples a random scalar share $\sigma_u^i \leftarrow \mathbb{Z}_p$ and gives $\{\sigma_u^i\}_{i\in[B]}$ to party u. For each honest party $j \notin S$ and each slot $i \in [B]$, let f_i denote the unique degree-$(t-1)$ polynomial with

$$f_i(0) = \tau^i \qquad \text{and} \qquad f_i(u) = \sigma_u^i \text{ for all } u \in S.$$

For corrupted parties, define

$$\widehat{\sigma}_u^i := \left[\sigma_u^i\right]_1 \qquad \text{and} \qquad v_u^i := \left[\sigma_u^i\right]_2.$$

Although $\mathcal{B}$ does not know $f_i(j)$ as a scalar for honest parties $j \notin S$, it can compute the corresponding group elements

$$\widehat{\sigma}_j^i := [f_i(j)]_1 \qquad \text{and} \qquad v_j^i := [f_i(j)]_2$$

by interpolation in the exponent from $g_i = \left[\tau^i\right]_1$, $h_i = \left[\tau^i\right]_2$, and the known values $\widehat{\sigma}_u^i = \left[\sigma_u^i\right]_1$ and $v_u^i = \left[\sigma_u^i\right]_2$ for $u \in S$. The reduction then sets

$$\mathsf{dk} := \left(\{h_j\}_{j\in[2B]\setminus\{B+1\}},\ \{v_j^i\}_{(i,j)\in[B]\times[N]},\ \mathsf{crs}\right).$$

To answer a PartialDec query on a batch $\{\mathsf{ct}_\ell\}_{\ell\in[B]}$, $\mathcal{B}$ first checks whether the query contains the challenge ciphertext ct^*. If so, it returns $\perp$ for that query, exactly as in the B-IND-CCA game definition. Otherwise, $\mathcal{B}$ verifies all NIZK proofs. If any proof is invalid, it returns $\perp$, exactly as the real scheme does. Otherwise, for each ciphertext $\mathsf{ct}_\ell = (\mathsf{ct}_{\ell,1}, \mathsf{ct}_{\ell,2}, \pi_\ell)$, straight-line simulation-extractability yields a witness k_ℓ such that $\mathsf{ct}_{\ell,1} = [k_\ell]_1$. For each honest party $j \notin S$, the reduction returns

$$\mathsf{pd}_j := \sum_{\ell\in[B]} k_\ell \cdot \widehat{\sigma}_j^\ell = \sum_{\ell\in[B]} [k_\ell f_\ell(j)]_1 \in \mathbb{G}_1,$$

which is exactly the honest partial decryption share.

When $\mathcal{A}$ outputs its challenge request, $\mathcal{B}$ responds with

$$\mathsf{ct}^* := (K,\ m_0 + Z,\ \pi^*) \qquad \text{where} \qquad \pi^* \leftarrow \mathsf{SimProve}_{\mathsf{NIZK}}(\mathsf{td}, (K, m_0 + Z)).$$

That is, π^* is a simulated proof for the relation

$$\{k \mid K = [k]_1 \wedge m_0 + Z\}.$$

If $Z = \left[k\tau^{B+1}\right]_T$, then this is distributed exactly as the challenge ciphertext in $\mathbf{H}_2$; if Z is uniform in $\mathbb{G}_T$, then $m_0 + Z$ is also uniform in $\mathbb{G}_T$, so this is distributed exactly as $\mathbf{H}_3$. Therefore a distinguisher between $\mathbf{H}_2$ and $\mathbf{H}_3$ yields a distinguisher for the q-DBPDH assumption, leading to contradiction. Hence $\mathbf{H}_2$ and $\mathbf{H}_3$ are computationally indistinguishable.

Finally, $\mathbf{H}_3$ is message-independent: if U is uniform in $\mathbb{G}_T$, then both $m_0 + U$ and $m_1 + U$ are uniform in $\mathbb{G}_T$ as well. Hence replacing $m_0 + U$ by $m_1 + U$ does not change the distribution of the challenge ciphertext in $\mathbf{H}_3$. Starting from this identical message-independent hybrid, we can apply the same hybrid transitions in reverse with m_1 in place of m_0, obtaining the real game in which the challenge ciphertext encrypts m_1. Therefore the real games for challenge bits 0 and 1 are computationally indistinguishable, so the construction is B-IND-CCA-secure. □

6 FFT Accelerated Decryption

The decryption procedure in Fig. 1 requires $O(B^2)$ pairings to decrypt a batch of size B if implemented directly. We can reduce this to $O(B)$ pairings and $O(B \log B)$ group operations by exploiting the Toeplitz structure of the masking terms via FFT-style convolution. The expensive part of decryption is computing the family of cross-terms

$$C_i := \sum_{\substack{\ell\in[B] \\ \ell\neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i} \qquad \text{for } i \in [B].$$

This is a Toeplitz correlation: the $\mathbb{G}_2$ index depends only on the difference $\ell - i$. That structure lets us compute all C_i simultaneously by an FFT-style evaluation/interpolation procedure, rather than recomputing each sum independently.

For simplicity, assume B is a power of two and that $\mathbb{F}_p$ contains a primitive $2B$-th root of unity ω. We use transform length $L = 2B$. Define the $\mathbb{G}_1$-valued sequence

$$a_i := \begin{cases} \mathsf{ct}_{i,1} & \text{for } i \in [B], \\ 0 & \text{otherwise,} \end{cases}$$

and the $\mathbb{G}_2$-valued sequence indexed by shifts $d \in \{-(B-1), \ldots, B-1\}$,

$$b_d := \begin{cases} h_{B+1+d} & \text{if } d \neq 0, \\ 0 & \text{if } d = 0, \end{cases}$$

again padded by zeros to length $2B$. Then

$$C_i = \sum_{\ell \in [B]} a_\ell \circ b_{\ell - i},$$

where the $d = 0$ term vanishes because we set $b_0 = 0$.

Now take the DFTs of these sequences, with coefficients in the source groups:

$$\widehat{A}_r := \sum_{i=0}^{L-1} \omega^{ri} a_i \in \mathbb{G}_1, \qquad \widehat{B}_r := \sum_{d=0}^{L-1} \omega^{-rd} b_d \in \mathbb{G}_2, \qquad r \in \{0, \ldots, 2B-1\}.$$

By bilinearity,

$$\widehat{C}_r := \widehat{A}_r \circ \widehat{B}_r = \sum_{i,d} \omega^{r(i-d)} \big(a_i \circ b_d\big) \in \mathbb{G}_T.$$

Applying the inverse DFT in $\mathbb{G}_T$ recovers

$$C_i = \frac{1}{2B} \sum_{r=0}^{2B-1} \omega^{-ri} \widehat{C}_r \qquad \text{for each } i \in [B].$$

So all cross-terms can be computed from:

- one FFT over $\mathbb{G}_1$ to form the $\widehat{A}_r$ values,
- one FFT over $\mathbb{G}_2$ to form the $\widehat{B}_r$ values,
- $2B = O(B)$ pairings to compute the $\widehat{C}_r$ values,
- one inverse FFT over $\mathbb{G}_T$,
- B pairings to compute $\mathsf{pd} \circ h_{B+1-i}$ for each $i \in [B]$.

The total cost is $O(B)$ pairings and $O(B \log B)$ group operations. Once the C_i are available, one can recover the messages as

$$z_i = \mathsf{pd} \circ h_{B+1-i} - C_i \qquad \text{and} \qquad m_i = \mathsf{ct}_{i,2} - z_i$$

for all $i \in [B]$.

7 Implementation and Benchmarks

To evaluate the concrete performance of our scheme, we implemented it in Rust, using arkworks [ac22] for implementations of pairing-friendly curves. Our implementation can be found at https://github.com/commonwarexyz/simple-bte.

Setup. Experiments were run on an M5 MacBook Pro with 48 GB of RAM in single-threaded mode. We use BLS12-381 [BLS03] as our pairing friendly curve. The compressed representation of $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ elements is 48, 96, and 576 bytes respectively. Group exponentiations in $\mathbb{G}_1$ and $\mathbb{G}_2$ take approximately 68 μs and 193 μs respectively, and a pairing takes approximately 443 μs.

Benchmarks. We now discuss microbenchmarks for various algorithms in our scheme.

- **Encryption.** Always takes approximately 0.47 ms irrespective of batch size and the number of parties.
- **NIZK Verify.** Verifies that a batch of ciphertexts is valid using batch verification of Schnorr proofs [APB+04] which can be carried out using $O(B)$ hashes and an MSM of size $O(B)$.
- **PartialDec.** Computes the partial decryption share for a given party using an MSM of size $O(B)$.
- **Verify.** Verifies a partial decryption share, by computing a multi-pairing of size $O(B)$.
- **Combine.** Combines a set of partial decryption shares into a final decryption by computing Lagrange coefficients and using an MSM of size $O(N)$.
- **Decryption.** Decrypts a batch of ciphertexts using a $\mathbb{G}_1$-FFT of size $O(B)$, a $\mathbb{G}_T$-FFT of size $O(B)$, and $O(B)$ pairings.

Operation	$B = 8$	$B = 32$	$B = 128$	$B = 512$	$B = 2048$
PartialDec (ms)	0.832	2.208	5.963	18.40	59.55
Verify (ms)	1.508	4.074	14.61	55.87	222.66
Combine (ms)	0.678	0.678	0.681	0.676	0.677

Comparison with prior work. We compare our batch decryption times against two prior schemes: [ABD+25], which achieves $O(B \log B)$ decryption but lacks censorship resistance, and TrX [FPTX25], which offers censorship resistance but has $O(B \log^2 B)$ decryption complexity. We do not compare against [BNRT26] as it has no public implementation. The [ABD+25] numbers are the core protocol times from their Table 3 (unweighted, NIZK overhead excluded). The [FPTX25] numbers are the sum of Digest, ComputeEvalProofs, and Decrypt from their Table 1.

Batch size B	Ours	TrX [FPTX25]	[ABD+25]
32	121.81 ms	100.7 ms	301 ms
128	593.62 ms	638.4 ms	1.4 s
512	2.79 s	5.39 s	6.6 s
2048	12.82 s	46.2 s	28.4 s

8 Acknowledgments

We thank Patrick O'Grady and Lucas Meier for their valuable feedback.

References

[ABD+25] Amit Agarwal, Kushal Babel, Sourav Das, Babak Poorebrahim Gilkalaye, Arup Mondal, Benny Pinkas, Peter Rindal, and Aayush Yadav. Weighted batched threshold encryption with applications to mempool privacy. Cryptology ePrint Archive, Paper 2025/2115, 2025. 4, 5, 12

[ac22] arkworks contributors. `arkworks` zksnark ecosystem. https://arkworks.rs, 2022. 11

[AFP25] Amit Agarwal, Rex Fernando, and Benny Pinkas. Efficiently-thresholdizable batched identity based encryption, with applications. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 69–100. Springer, Cham, August 2025. 4, 5

[APB+04] Riza Aditya, Kun Peng, Colin Boyd, Ed Dawson, and Byoungcheon Lee. Batch verification for equality of discrete logarithms and threshold decryptions. In Markus Jakobsson, Moti Yung, and Jianying Zhou, editors, *ACNS 2004*, volume 3089 of *LNCS*, pages 494–508. Springer, Berlin, Heidelberg, June 2004. 12

[BCF+25] Jan Bormet, Arka Rai Choudhuri, Sebastian Faust, Sanjam Garg, Hussien Othman, Guru-Vamsi Policharla, Ziyan Qu, and Mingyuan Wang. BEAST-MEV: Batched threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Paper 2025/1419, 2025. 5

[BFOQ25] Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyan Qu. BEAT-MEV: Epochless approach to batched threshold encryption for MEV prevention. In Lujo Bauer and Giancarlo Pellegrino, editors, *USENIX Security 2025*, pages 3457–3476. USENIX Association, August 2025. 4, 5

[BGW05] Dan Boneh, Craig Gentry, and Brent Waters. Collusion resistant broadcast encryption with short ciphertexts and private keys. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 258–275. Springer, Berlin, Heidelberg, August 2005. 1

[BLS03] Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. Constructing elliptic curves with prescribed embedding degrees. In Stelvio Cimato, Clemente Galdi, and Giuseppe Persiano, editors, *SCN 02*, volume 2576 of *LNCS*, pages 257–267. Springer, Berlin, Heidelberg, September 2003. 11

[BNRT26] Dan Boneh, Rohit Nema, Arnab Roy, and Ertem Nusret Tas. Efficient batch threshold encryption using partial fraction techniques. Cryptology ePrint Archive, Paper 2026/674, 2026. 5, 12

[CGPP24] Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024. 1, 4

[CGPW25] Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. Practical mempool privacy via one-time setup batched threshold encryption. In Lujo Bauer and Giancarlo Pellegrino, editors, *USENIX Security 2025*, pages 3477–3495. USENIX Association, August 2025. 4, 5

[FPTX25] Rex Fernando, Guru-Vamsi Policharla, Andrei Tonkikh, and Zhuolun Xiang. TrX: Encrypted mempools in high performance BFT protocols. Cryptology ePrint Archive, Report 2025/2032, 2025. 4, 5, 12

[GWWW25] Junqing Gong, Brent Waters, Hoeteck Wee, and David J. Wu. Threshold batched identity-based encryption from pairings in the plain model. Cryptology ePrint Archive, Report 2025/2103, 2025. 5

[SAS24] Sora Suegami, Shinsaku Ashizawa, and Kyohei Shibano. Constant-cost batched partial decryption in threshold encryption. Cryptology ePrint Archive, Paper 2024/762, 2024. 4, 5